ADR Template (tham khảo thêm: Link)

Architecture Decision Record (ADR) dùng để ghi lại **quyết định kiến trúc quan trọng**, lý do ra quyết định, các phương án đã cân nhắc, trade-offs (đánh đổi), tác động và thời điểm cần xem xét lại.

Mục tiêu:

- Giúp team hiểu **vì sao quyết định được đưa ra**
- Tránh tranh luận lặp lại
- Giảm technical debt (nợ kỹ thuật) vô thức
- Hỗ trợ onboarding (người mới vào team)
- Tăng governance / traceability (khả năng kiểm soát & truy vết quyết định)

Template này phù hợp cho nhiều level khác nhau:

- Engineer / Junior có thể dùng để học cách tư duy ra quyết định có cấu trúc.
- Senior Engineer / Tech Lead dùng để ghi lại và truyền đạt các quyết định kỹ thuật quan trọng.
- Architecture / Engineering Manager dùng để alignment stakeholder và governance ở quy mô lớn.

Tôi cố tình thiết kế nó không chỉ là biểu mẫu điền thông tin, mà là một framework giúp mọi người suy nghĩ đúng trước khi quyết định.

Cách sử dụng template này

Khi nào nên viết ADR?

Viết ADR khi quyết định có ảnh hưởng đáng kể như:

- Chọn kiến trúc Monolith / Microservices
- Chọn DB (PostgreSQL / MongoDB)
- Chọn multi-tenant strategy (chiến lược nhiều khách hàng dùng chung hệ thống)
- Chọn auth model (mô hình xác thực)
- Chọn event-driven vs sync API
- Chọn cloud provider
- Quyết định có chi phí đảo ngược cao

Khi nào không cần ADR?

Không cần ADR cho thay đổi nhỏ:

- đổi tên field
- chỉnh màu button
- sửa bug nhỏ
- refactor nội bộ không ảnh hưởng design

Quy tắc vàng

Quyết định càng khó đảo ngược, càng nên có ADR.

ADR-00X: [Tên quyết định ngắn gọn, rõ ràng, dùng động từ]

Cách đặt tên tốt

Dùng động từ + đối tượng:

- Adopt PostgreSQL as Primary Database
- Use Modular Monolith for Phase 1
- Standardized Event Bus with Kafka
- Isolate Tenant Data by Schema-per-Tenant

Tên kém chất lượng

- Database Decision
- Architecture Change
- Some Improvements

Metadata

- **Status:** Proposed | Accepted | Rejected | Superseded | Deprecated
  - **Proposed** = đang đề xuất
  - **Accepted** = đã được chấp thuận
  - **Rejected** = đã phân tích nhưng không chọn
  - **Superseded** = bị thay thế bởi ADR mới hơn
  - **Deprecated** = không còn nên dùng

Hướng dẫn:

- Chỉ 1 trạng thái chính thức
- Khi thay đổi phải cập nhật lịch sử

- **Date (ngày quyết định):** YYYY-MM-DD
  - Ngày ra quyết định chính thức.

- **Owner (người chịu trách nhiệm):** [Người / Team]
    - Người trả lời câu hỏi về ADR này.
    - Ví dụ:
        - Platform Team
        - Tech Lead - Payments Domain

- **Decision Scope (phạm vi ảnh hưởng):** System | Domain | Service | Platform | Org-wide
    - **System** = toàn hệ thống
    - **Domain** = một domain nghiệp vụ
    - **Service** = một service cụ thể
    - **Platform** = hạ tầng / nền tảng chung
    - **Org-wide** = ảnh hưởng toàn công ty

- **Decision Type (loại quyết định):** Strategic | Tactical | Operational
    - **Strategic** = dài hạn, khó đảo ngược
    - **Tactical** = trung hạn, tối ưu thực thi
    - **Operational** = phục vụ vận hành ngắn hạn
    - Ví dụ:
        - Strategic: chuyển monolith sang modular monolith
        - Tactical: chọn Redis làm cache
        - Operational: đổi cron scheduler tool

- **Priority (mức ưu tiên):** Critical | High | Medium | Low
    - Critical = cần xử lý gấp, ảnh hưởng lớn
    - High = quan trọng

    - Medium = nên làm
    - Low = có thể trì hoãn

- **Related ADRs (ADR liên quan):** ADR-001, ADR-014
- **Supersedes (thay thế ADR cũ):** ADR-00Y
- **Tags (nhãn tìm kiếm):** scalability, security, maintainability

---

1. Executive Summary (Tóm tắt thực hiện)

Hướng dẫn viết:

- Trả lời 4 câu:
    - Vấn đề gì?
    - Đề xuất gì?
    - Vì sao chọn?
    - Kỳ vọng gì?
- Ví dụ:
    - Hệ thống hiện deploy chậm và khó scale.
    - Đề xuất chuyển sang containerized deployment trên Kubernetes.
    - Giải pháp tăng complexity (độ phức tạp) vừa phải nhưng giảm downtime.
    - Kỳ vọng deploy time giảm từ 40 phút còn 10 phút.

---

2. Context / Problem Statement (Bối cảnh / Bài toán)

- Đây là phần quan trọng nhất.

- Nếu context sai thì decision đúng kỹ thuật vẫn có thể sai tổng thể.

2.1 Current State (Hiện trạng)

- Mô tả hiện trạng:
    - Quy mô user: …
    - Peak traffic (lưu lượng cao điểm): …
    - Team size: …
    - Kiến trúc hiện tại: …
    - Giai đoạn business: …
- Ví dụ:
    - **Quy mô user:** 500k monthly users
    - **Peak traffic (lưu lượng cao điểm):** Peak 2k RPS (request/giây)
    - **Team size:** Team 8 engineers
    - **Kiến trúc hiện tại:** Monolith + PostgreSQL
    - **Giai đoạn business:** Growth stage

2.2 Business Need (Nhu cầu kinh doanh)

Business thật sự cần gì?

- Ví dụ:
    - Release nhanh hơn
    - Hỗ trợ enterprise customer
    - Multi-region
    - Compliance SOC2
    - Giảm burn rate (tốc độ đốt tiền)

ADR yếu thường chỉ nói tech, quên business.

2.3 Current Pain Points (Vấn đề hiện tại)

Hướng dẫn:

- Nên có số liệu.
- Ví dụ:
    - Deploy mất 45 phút
    - MTTR = 2 giờ
    - 6 incidents / tháng
    - Onboarding dev mới mất 3 tuần

MTTR = Mean Time To Recovery (thời gian phục hồi trung bình)

2.4 Constraints (Ràng buộc)

Các điều không thể bỏ qua.

- Ví dụ:
    - Deadline 6 tuần
    - Không tăng headcount
    - Budget cloud tối đa $5k/tháng
    - Không breaking API public

2.5 If We Do Nothing (Nếu không làm gì)

- mất khách hàng
- debt tăng

- tốc độ delivery giảm
- risk cao hơn

Đây là cost of inaction (chi phí của việc không hành động).

2.6 Why Now? (Tại sao là lúc này)

- Ví dụ:
    - sắp ký khách hàng lớn
    - hạ tầng sắp quá tải
    - audit sắp tới

---

3. Stakeholders & Concerns (Bên liên quan & Mối quan tâm)

- Stakeholder = bên liên quan
- Concern = điều họ quan tâm nhất

Stakeholder	Vai trò	Concern	Success Criteria
Business	Revenue owner	Time to market	Release < 2 weeks
Product	Delivery owner	Predictability	Stable roadmap
Dev Team	Builder	Maintainability	Easy onboarding

Ops/SRE	Reliability	Stability	99.9% uptime

Hướng dẫn:

- Mỗi stakeholder có thể mâu thuẫn nhau:
    - Business muốn nhanh
    - Security muốn chặt
    - Finance muốn rẻ

ADR tốt phải cân bằng.

---

4. Decision Drivers (Tiêu chí ra quyết định)

- Những yếu tố quan trọng dùng để so sánh các phương án.

Driver	Priority	Meaning
Simplicity	High	dễ hiểu, dễ vận hành
Scalability	High	scale tốt
Cost	Medium	chi phí hợp lý
Security	High	giảm risk

Hướng dẫn:

- Nếu mọi thứ đều High => chưa có ưu tiên thật.

---

5. Considered Options (Các phương án đã cân nhắc)

- Luôn ghi nhiều phương án để tránh “decision made first, document later”.

Option A: Keep Monolith

Description (Mô tả):

- Giữ monolith, tối ưu codebase hiện tại.

Pros (Ưu điểm):

- Không migration lớn
- Team quen thuộc

Cons (Nhược điểm):

- Khó scale team ownership

Risks (Rủi ro):

- Debt tiếp tục tăng

Cost Estimate (Ước tính chi phí):

- Build Cost (chi phí triển khai):
- Run Cost (chi phí vận hành):
- Learning Cost (chi phí học):

Reversibility (Khả năng quay đầu/ đảo ngược quyết định)

- Easy | Medium | Hard

Option B: Modular Monolith

Description (Mô tả)

- Tách monolith thành module rõ ràng nhưng deploy chung.

Pros (Ưu điểm)

- Boundary tốt
- Complexity thấp hơn microservice

Cons (Nhược điểm)

- Cần discipline kiến trúc

Risks (Rủi ro)

- Nếu governance yếu sẽ regress (thoái lui chất lượng)

Option C: Microservices

Description (Mô tả)

- Tách thành nhiều service độc lập.

Pros (Ưu điểm)

- Team autonomy cao

Cons (Nhược điểm)

- Ops complexity cao

Risks (Rủi ro)

- Over-engineering sớm

---

6. Decision Matrix (Bảng so sánh quyết định)

Criteria	Weight	A	B	C
Simplicity	5	5	4	2
Scalability	5	2	4	5
Cost	4	5	4	2
Total	-	12	13	9

Thuật ngữ:

- **Criteria** = tiêu chí
- **Weight** = trọng số quan trọng
- **Score** = điểm đáp ứng

Đây là công cụ hỗ trợ reasoning (lý luận), không phải máy quyết định.

---

7. Decision Outcome (Kết quả quyết định)

Chosen Option (Phương án được chọn):

- **Option B: Modular Monolith**

Why This Option Wins (Vì sao thắng):

- Cân bằng scale và simplicity
- Team hiện tại đủ năng lực
- Time-to-market tốt hơn microservices

Why Not Others (Vì sao không chọn phương án khác):

- A không đủ cho tăng trưởng 12 tháng tới
- C quá phức tạp giai đoạn hiện tại

Decision Confidence (Mức độ tự tin):

- High | Medium | Low
- Nếu Low => cần benchmark / POC / experiment

---

8. Impact Across 4+1 Views (Tác động theo 4+1 View Model)

Use Case View (Nghiệp vụ / người dùng)

- Ví dụ:
    - onboarding nhanh hơn

Logical View (Domain / module)

- Ví dụ:
    - tách Billing khỏi User domain

Development View (Code / team ownership)

- Ví dụ:
    - team A sở hữu Payments module

Process View (Runtime behavior)

- Ví dụ:
    - queue riêng cho email jobs

Physical View (Infra / deployment):

- Ví dụ:
    - thêm autoscaling nodes

---

9. Consequences (Hệ quả)

Positive (Tích cực)

- deploy nhanh hơn
- boundary rõ hơn

Negative (Tiêu cực)

- cần training team
- migration cost tăng

Trade-offs Accepted (Đánh đổi chấp nhận)

- Thứ chấp nhận mất để đổi lấy thứ quan trọng hơn.
- Ví dụ:
    - tăng ops complexity để scale tốt hơn
    - tăng chi phí cloud để tăng reliability

---

10. Risks & Mitigations (Rủi ro & Giảm thiểu)

Risk	Severity	Probability	Mitigation
Migration delay	High	Medium	rollout theo phase
Skill gap	Medium	High	training
Cost spike	Medium	Medium	budget alerts

Thuật ngữ:

- **Severity** = mức độ nghiêm trọng
- **Probability** = khả năng xảy ra
- **Mitigation** = cách giảm thiểu

---

11. Rollout Plan (Kế hoạch triển khai)

Phase 1: Validation (Xác thực)

- POC
- baseline metrics

Phase 2: Controlled Rollout (Triển khai giới hạn)

- 5% traffic
- monitor logs/errors

Phase 3: Full Adoption (Triển khai toàn bộ)

- 100% traffic

Rollback Plan (Kế hoạch quay lui)

- Nếu lỗi lớn:
    - route traffic về bản cũ
    - restore snapshot
    - disable feature flag

---

12. Success Metrics (Chỉ số thành công)

Metric	Baseline	Target
Deploy time	40 min	10 min
p95 latency	450ms	<300ms
Incident/month	6	<2

Thuật ngữ:

- **Baseline** = mức hiện tại
- **Target** = mục tiêu
- **p95 latency** = 95% request nhanh hơn giá trị này

---

13. Revisit Trigger (Điều kiện xem lại)

> **“**ADR tốt không phải bất biến.**”**

- traffic > 5x

- team tăng gấp đôi
- regulation đổi
- cost +20%
- incidents lặp lại

---

14. Notes / Evidence (Ghi chú / Bằng chứng)

> "Ý kiến không bằng dữ liệu."

- benchmark
- prototype result
- incident report
- vendor comparison
- architecture diagram

---

15. Anti-Patterns Checklist (Dấu hiệu ADR tệ)

- **Không approve nếu:**
  - chỉ theo trend
  - không so sánh option
  - không rollback plan
  - không metric
  - over-engineering
  - team không đủ skill

---

16. Final Approval (Phê duyệt cuối)

Role	Name	Decision
Tech Lead	...	Approve
EM	...	Approve
Product	...	Acknowledge

“Acknowledge = đã biết tác động, không nhất thiết là owner kỹ thuật.”

Writing Guidance (Hướng dẫn viết)

1. **ADR tốt là:**
    - ngắn nhưng sắc
    - rõ trade-off
    - có dữ liệu
    - gắn business value
    - có revisit trigger
2. **ADR tệ là:**
    - viết để khoe tech
    - dài nhưng rỗng
    - không ai chịu trách nhiệm
    - không dùng lại được

Golden Rule

- “Không có quyết định hoàn hảo.”
- “Chỉ có quyết định phù hợp nhất với context hiện tại.”

“(Chỉ có quyết định phù hợp nhất với context, timing và constraints hiện tại.)”

Nhật ký cập nhật / Changelog (Optional):

- Có ích nếu ADR lưu trên Wiki / Notion / Docs.
- Nếu ADR quản lý bằng Git tốt, phần này là tùy chọn.

Date	Change	By
2025-01-01	Status updated from Proposed to Accepted	Tech Lead
2025-01-02	Added Option C: Managed Kafka	Platform Team
2025-01-03	Revised cost estimates after vendor review	Finance + Engineering
2025-01-05	Superseded by ADR-021	Architecture Board

- Nhiều template ADR phổ biến chỉ tập trung ghi lại kết quả quyết định.
- Template của tôi bổ sung thêm phần hướng dẫn tư duy như context, decision drivers, reversibility, metrics, stakeholder concerns và revisit trigger.

Hiểu đơn giản là:

Không chỉ dừng lại ở việc “***đã chọn gì***”,

Mà giúp chúng ta hiểu “***vì sao chọn***” và “***khi nào cần xem lại***”.

Mr. Đức

duch9707@gmail.com | linkedin.com/in/mr-duc